

Secure Programming: Exercise 3

Due Date: June 02, 2026

General Instructions

- Work individually or in pairs only. You may use internet resources to solve the tasks, but you must cite the sources you used.
- In any case, you must reach the final solution independently.
- For any script you write to solve the exercise: Include a comment at the top explaining why you wrote the script and how it assisted you in finding the solution.
- Scripts must be written in `Python 3.8` without non-trivial dependencies.
- The exercise is evaluated using a Chrome-based browser. It is expected to work on any modern browser, but it has not been specifically tested on others (such as Firefox or Safari).
- In all sections (except Section A), **do not look at the source code itself** (assume the website is live on the internet and you do not have direct access to its code).

Exercise

You are provided with a website package named `XSSApp-1.0.tar.gz`. The website is a Python (3.8) package that can be installed and then run locally.

- You can run it using `python -m XSSApp` or by executing `start_xss_app.exe` (on other operating systems, it will have a different extension).
- Verify that the website is functioning. Read the website's code—it is located inside the zip package.
- Try to log in as an administrator (the password is in the code), and try to log out. Essentially, explore all the features (there are not many).
- **Recommendation:** As a starting point, extract the source directory found inside `XSSApp` to your workspace.

Tasks

- A. Add a digital certificate (signature) to your website (where the signature is intended for a site with a logical/valid name), so that it can be browsed using `HTTPS`, and in a way that your browser recognizes the site's certificate as secure.**
- You must ensure that the Chrome browser (version 85 and above) also recognizes your certificate as secure.

- Detail exactly how you managed to add the signature to the site (write a tutorial so that someone else can follow it and replicate this section).
- Include a screenshot showing you running the site securely.

B. Find an XSS vulnerability on the site. Exploit it to impersonate a highly privileged user.

- Demonstrate how you delete all messages on the server using a low-privileged ("weak") user.

C. Execute code using a valid HTML `<object>` tag. Attempt to steal cookies using it. This operation is expected to fail.

- Explain why the operation failed.

D. The website contains a fair amount of security vulnerabilities and violates accepted guidelines for writing secure code.

- Find at least 3 areas where you could improve the site's security, *excluding* the BeautifulSoup vulnerability.

E. Explain how you would successfully execute the attack from Section B even if the session key was configured as `HttpOnly`.

F. (Optional) Explain what XST is.

G. (Optional) Explain what UXSS is and how it differs from regular XSS. Provide at least one use-case where UXSS can be achieved.

- Detail whether UXSS can be achieved without exploiting a vulnerability in the browser itself.

Hints for the Exercise

- `BeautifulSoup` does not accept broken HTML tags, whereas Chrome-based browsers do allow them. A vulnerability is hidden within this discrepancy (there are many other vulnerabilities, though some are likely harder to exploit).
- `<OBJECT TYPE="text/x-scriptlet">`
- For **Section B**, it is highly recommended to write an attacking server ("backend"), rather than attacking exclusively from the client side.
- Use Python's `requests` module—it is very convenient for this.

Recommended Links

- **How to move the site to HTTPS:**

<https://www.youtube.com/watch?v=b3zeG6mQMWc>

- **Tags and attributes for XSS evasion:**
[OWASP XSS Filter Evasion Cheat Sheet](#)
[GitHub Gist: XSS Payloads](#)
- **Vulnerable sites for XSS practice:**
<http://sudo.co.il/xss/level0.php>
<https://prompt.ml/0>

Submission Guidelines

- The exercise must be submitted via the Learning Management System (LMS).
- The core of the submission should be a **PDF file** explaining how you solved each section of the exercise.
- You must attach all code snippets you wrote, along with explanations (points will be deducted if missing).
- Additionally, attach any files you generated during any of the sections (e.g., files created for signing the certificate or executing the attack).
- File names must be in English, containing **ASCII-encoded characters only**. Do not upload unnecessary or excessively large files.
- Every submission must be accompanied by a `details.txt` file containing the ID number and username of the submitter in the following format:
`Israel Israeli 123456789`
`Israel unisraeli 234567890`
- Good luck!